

Package model

Il package model contiene 6 classi che rappresentano le entità del sistema.

Libro.java è la classe base della gerarchia. Contiene gli attributi comuni a qualsiasi libro: **nomeLibro**, **nomeAutore**, **ISBN**, **numeroPagine**, **annoRilascio**, **prezzoArticolo** e **Categoria**, tutti pubblici. Ha un attributo privato boolean **disponibile** inizializzato a **true** che indica se il libro può essere acquistato o preso in prestito. Il costruttore accetta sette parametri e assegna i valori ai campi corrispondenti. La classe fornisce getter per tutti gli attributi, i metodi **isDisponibile()** e **setDisponibile()** per gestire la disponibilità, e il metodo **verificaDisponibilita()** che lancia un'eccezione di tipo **LibroNonDisponibileException** se il libro non è disponibile (se è terminato).

Cartaceo.java estende Libro e rappresenta un libro fisico. Aggiunge un solo attributo pubblico **tipoCopertina** di tipo String che indica se la copertina è Rigida o Flessibile. Il costruttore riceve gli otto parametri (sette per Libro più **tipoCopertina**) e chiama **super()** per inizializzare la parte ereditata.

EBook.java estende Libro e rappresenta un libro digitale. Aggiunge un unico attributo pubblico **formatoFile** di tipo String che indica il formato del file digitale (PDF, Kindle o HTML). Il costruttore ha la stessa struttura di Cartaceo: riceve otto parametri e chiama **super()** per la parte ereditata.

Tessera.java rappresenta la tessera fedeltà del cliente. Ha due campi privati: **id** e **tipo**, dove **tipo** può essere Standard o Studente. Il metodo **isValida()** controlla che l'id non sia nullo o vuoto. Il metodo **getSconto()** restituisce 0.15 per le tessere Studente e 0.10 per le

tessere Standard. Il metodo **applicaSconto()** riceve un prezzo e restituisce il prezzo ridotto applicando la percentuale di sconto.

Cliente.java rappresenta un utente della libreria. Ha tre campi privati: **nomeCliente**, **cognomeCliente** e tessera di tipo **Tessera**. Il costruttore inizializza nome e cognome, mentre la tessera parte a null e viene assegnata successivamente tramite **setTessera()**.

Libreria.java rappresenta il negozio fisico. Ha i campi **nomeNegozio**, **indirizzoNegozio**, **orarioApertura** e **orarioChiusura** di tipo **LocalTime**, e una lista di clienti di tipo **ArrayList**. Il metodo **aperta()** confronta l'ora corrente con gli orari di apertura e chiusura per determinare se il negozio è aperto. Il metodo **aggiungiCliente()** aggiunge un cliente alla lista.

Package exception

LibroNonDisponibileException.java è un'eccezione **checked** che estende **Exception**. Viene lanciata quando si tenta di acquistare o prendere in prestito un libro non disponibile. Il costruttore riceve il nome del libro e compone un messaggio di errore descrittivo che viene passato alla superclasse tramite **super()**. Il campo privato **nomeLibro** è accessibile tramite il getter **getNomeLibro()**.

Package controller

Il package controller contiene due classi che gestiscono la logica delle operazioni.

Acquisto.java rappresenta una transazione di acquisto. Ha cinque campi privati: **numeroTransazione** di tipo **String**, **dataAcquisto** di

tipo **LocalDate**, **oraAcquisto** di tipo **LocalTime**, metodo di tipo String per il metodo di pagamento, e prezzo di tipo double. Il metodo **pagamentoFinale()** restituisce il prezzo. Il metodo **getScontrino()** genera una stringa formattata contenente tutti i dati della transazione.

Prestito.java rappresenta un'operazione di prestito. Ha tre campi privati: **ricevutaPrestito** di tipo String, **inizioPrestito** e **finePrestito** di tipo **LocalDate**. Il metodo **getRicevuta()** restituisce il codice della ricevuta. Il metodo **restituzione()** confronta la data odierna con la data di fine prestito e restituisce true se il prestito è scaduto.

Package gui

Il package gui contiene tre classi che compongono l'interfaccia grafica.

Login.java gestisce la schermata di accesso. Mostra due campi di testo per nome e cognome, un dropdown per scegliere il tipo di tessera (Standard o Studente) e due bottoni Accedi e Registrati. Quando l'utente preme uno dei bottoni, il metodo **eseguiLogin()** controlla che i campi non siano vuoti, crea un oggetto Cliente con la sua Tessera e notifica la HomeLibreria tramite una callback di tipo LoginListener. Il metodo statico **apri()** crea la finestra e la mostra.

HomeLibreria.java è la schermata principale. I componenti grafici vengono creati dal file .form di IntelliJ ma il layout viene ricostruito da codice nel costruttore usando un BorderLayout: in alto la barra di ricerca con il filtro per categoria, al centro la lista dei libri dentro uno JScrollPane, in basso la label dell'account e il bottone Login/Logout. La classe mantiene un ArrayList catalogo con tutti i libri e un ArrayList risultatiCorrenti con i libri filtrati. Il metodo **cercaLibri()** filtra il catalogo in base al testo e alla categoria selezionata e aggiorna la lista. Il metodo **aggiornaStatoLogin()** gestisce il passaggio tra lo

stato Ospite e lo stato loggato. Il doppio click su un libro della lista apre la finestra di dettaglio.

InterfacciaLibro.java mostra i dettagli di un singolo libro: autore, ISBN, categoria, tipo, pagine, anno, prezzo con eventuale sconto della tessera e stato della disponibilità. In basso ha due bottoni. Il bottone Acquista e il bottone Prestito usano entrambi un blocco try-catch: chiamano **libro.verificaDisponibilita()** e se il libro è disponibile completano l'operazione, altrimenti catturano l'eccezione **LibroNonDisponibileException** e mostrano un messaggio di errore. Il prestito è consentito solo per i libri Cartacei. Il metodo statico **apri()** crea la finestra e la mostra.

Package Main

Main.java è il punto di ingresso dell'applicazione. Il metodo **main()** crea un oggetto Libreria con nome, indirizzo e orari di apertura. Crea un ArrayList di tipo Libro chiamato catalogo e vi inserisce i libri alternando oggetti Cartaceo e EBook distribuiti sulle categorie Romanzo, Informatica, Cucina, Horror, Thriller, Fantasy, Avventura, Fantascienza, Narrativa e Romantico ed etc. Avvia la GUI all'interno di **SwingUtilities.invokeLater()** per garantire l'esecuzione sul thread dedicato alla grafica: crea il JFrame principale con il titolo della libreria, istanzia la HomeLibreria, aggiunge tutti i libri del catalogo tramite un ciclo for-each, e rende visibile la finestra.